



Rapport projet d'informatique – Groupe 9

Auteur.e.s :

Bou Khaled Yasmina

Dejean Romain

Godeau Gauthier

17 décembre 2025

Enseignant :

Jérémie Roland

1 Introduction

Dans le cadre du cours de complément de programmation et d'algorithmique, il a été demandé d'implémenter un programme capable de lire en entrée une séquence de protéine à aligner avec une grande base de données de séquences en utilisant l'algorithme de Smith-Waterman. Ce rapport décrit la réalisation de ce projet en s'attardant particulièrement sur la structure du programme remis sur le dépôt git.

2 Fonctions et classes implémentées

Notre projet est décomposé en plusieurs fichiers de la manière suivante :

2.1 Main.cpp

Ce fichier est à la base de notre code, son but est de renvoyer un fichier texte listant les vingt protéines les mieux alignées avec la protéine de requête en faisant appel à différentes fonctions présentées ci-dessous.

2.2 Lecture_fasta.cpp

Ce fichier contient la fonction *lect_fasta*. Cette fonction prend en entrée le chemin du fichier binaire *.fasta* contenant la séquence de la protéine de requête. Ce fichier est ouvert puis parcouru ligne par ligne. Chaque ligne est stockée successivement dans un string « line » qui est agrandi à chaque fois pour obtenir la séquence complète de la protéine de requête. Celle-ci est renvoyée en sortie de la fonction sous forme de string.

2.3 Lecture_blosum.cpp

Ce fichier contient la fonction *lect_blosum*. Cette fonction prend en entrée le chemin vers le fichier texte « BLOSUM62 » contenant la matrice blosum et copie son contenu (c'est-à-dire les scores des différentes combinaisons d'acides aminés) dans la matrice attribut de la classe *BlosumMatrix*. Les lignes et les colonnes de la matrice correspondent à la traduction binaire de chaque acide aminé, comme décrit dans (M S Farrar - NCBI BLAST Database Format 2010) présent dans le dossier « reference » du dépôt git du projet. Par exemple, le score de A et U est l'élément « matrice [1][25] ».

2.4 Lecture_psq.cpp

Ce fichier contient la fonction *lect_psq* qui reçoit en entrée le chemin de la matrice blosum, le chemin vers la base de données, la séquence de la protéine de requête, le gap open penalty et le gap extension penalty. Son premier rôle est de lire, à l'aide la fonction *decoder*, chaque séquence de protéine présente dans la base de données et de calculer son score d'alignement avec la séquence de requête grâce à la fonction *smith_waterman*. Une fois le score calculé, celui-ci est regroupé avec l'indice de la protéine dans une paire, qui est ensuite ajoutée à un tableau de 20 emplacements si son score est supérieur au plus petit élément du tableau. Ce dernier est trié grâce à la fonction *tri* et est ensuite renvoyé dans la fonction *main*.

2.5 Smith_waterman.cpp

Cette fonction prend en entrée 6 arguments : la matrice blosum, la séquence de protéine de requête, la séquence de protéine de la base de données pour laquelle nous voulons calculer le score, la longueur de la séquence de requête, la valeur du gap open penalty et du gap extension penalty. Elle renvoie le meilleur score correspondant au meilleur alignement entre les deux protéines. Pour calculer ce score, nous avons implémenté l'algorithme de Smith-Waterman expliqué dans les documents de référence en réalisant deux boucles *for*, l'une parcourant la séquence de la protéine de requête et l'autre la séquence provenant de la base de données. Ceci permet d'obtenir tous les alignements possibles et trouver le meilleur score.

2.6 Lecture_pin.cpp

Ce fichier contient la fonction *lect_pin* qui reçoit en entrée le chemin vers la base de données et l'indice de la protéine dans la base de données. En connaissant cet indice, la fonction pourra chercher dans la « header offset table » la position de la protéine et de ses informations dans le fichier *.phr*. Une fois trouvée, cette position est renvoyée dans la fonction *main*.

2.7 Lecture_phr.cpp

Ce fichier contient la fonction *lect_phr*. Cette fonction permet de retrouver le nom de la protéine à partir de sa position (calculée avec la fonction *lect_pin*). Elle prend donc en entrée le chemin vers la base de données et la position de la protéine dans le fichier. La fonction parcourt le fichier jusqu'à arriver à la position voulue. La lecture du « header » commence au premier octet de la chaîne de caractère. Le « header » est ensuite enregistré et renvoyé dans la fonction *main* et sera, par après, imprimé avec les autres protéines possédant les 20 meilleurs scores.

3 Optimisation du temps d'exécution

3.1 Lecture de fichier binaire

L'essentiel de la première partie du projet porte sur la lecture de fichier binaire. Cette lecture est importante à maîtriser étant donné le nombre de fichiers binaires distincts qui doivent être lus (*.fasta*, *.psq*, *.pin*, *.phr*). Dans un premier temps, le fichier binaire était parcouru de bout en bout à l'aide de la fonction « read ». Cependant, cela n'était pas optimal sachant que la fonction *seekg* est beaucoup plus adaptée lorsque l'on sait exactement où se trouve l'information dans ce fichier binaire, ce qui est le cas ici. D'autant plus que les fichiers binaires utilisés ici sont très volumineux, ce qui justifie encore plus l'utilisation de *seekg*. Cette modification a permis d'avoir une amélioration du temps d'exécution de 19% pour les tests préliminaires.

3.2 Récupération des 20 meilleures séquences

Une réflexion s'est portée sur la manière de trier les 20 meilleures séquences de protéines. À la base, le tableau contenant les séquences est initialisé à 0 partout, ce qui fait de lui un tableau déjà trié. Au fur et à mesure les séquences vont être ajoutées une par une. À chaque ajout, le tableau est trié par un algorithme de tri. Cela signifie que lors de l'application de l'algorithme, le tableau est presque trié (19 séquences sur les 20 sont déjà triées entre elles). En analysant les différents

algorithmes de tri, il a rapidement été conclu que le tri par insertion était le plus adapté, étant donné que dans le cas où le tableau est presque trié, sa complexité est la meilleure, $O(n)$ et non $O(n \cdot \log(n))$.

Par la suite, afin de vérifier si cet algorithme était le meilleur, nous l'avons comparé avec le tri présent dans la bibliothèque standard C++. Après les tests, il s'est avéré que le tri par insertion fournissait un meilleur temps d'exécution en moyenne que le tri de la bibliothèque standard comme illustré au Tableau 1. C'est donc pour ces raisons que le choix de l'algorithme de tri par insertion est resté inchangé.

Une deuxième alternative était le conteneur dynamique. Cependant, cette alternative n'a pas été retenue pour plusieurs raisons. Afin de stocker seulement 20 protéines, il était préférable d'utiliser un tableau à qui la mémoire nécessaire a déjà été attribuée plutôt que d'utiliser un conteneur dont chaque insertion implique une allocation dynamique de la mémoire. De plus, les conteneurs dynamiques sont basés sur des arbres binaires de recherche, cela nécessite qu'à chaque insertion, 3 pointeurs, vers les enfants ainsi que leur parent, doivent être stockés. Cela rajoute beaucoup d'éléments à stocker en mémoire pour seulement 20 protéines sachant que les arbres binaires ne bénéficient pas d'une localité dans la mémoire. A l'inverse, un tableau garantit cette localité et permet de ne pas avoir autant de pointeurs à stocker.

Finalement, étant donné que le tri implémenté était très optimisé, il paraissait plus adéquat de procéder à l'aide d'un tableau et d'un tri plutôt que d'un conteneur ordonné.

Temps d'exécution avec le tri de la bibliothèque standard C++ (secondes)		Temps d'exécution avec le tri par insertion (secondes)
4,361		4,287
4,281		4,24
4,353		4,279
4,352		4,249
4,3		4,227
4,33		4,243
4,32		4,23
4,339		4,325
Moyenne	4,3295	4,26

Tableau 1 : Temps d'exécution en secondes en fonction du tri utilisé

3.3 Optimisation de smith_waterman

Vu que la fonction [smith_waterman](#) est appelée pour chaque protéine de la base de données, cela fait d'elle la fonction la plus appelée de ce projet. C'est donc cette fonction qui influence le plus le temps d'exécution du programme. Afin de réduire le nombre d'initialisations de variables dans la

fonction elle-même, ce qui reviendrait à initialiser plein de fois la même chose, toutes ces variables ont été mises en paramètres de la fonction permettant de ne les initialiser qu'une seule fois. C'est pour cette raison que la fonction possède un très grand nombre de variables. Pour le cœur de la fonction, l'objectif était de rendre le code le plus compact possible tout en limitant au maximum le nombre d'instructions par fonction. Tout cela a amené au code actuel de la fonction [*smith_waterman*](#).

Par ailleurs, toujours dans l'optique d'alléger l'algorithme de Smith Waterman, une classe *BlosumMatrix* a été créée afin de se charger de tous les aspects de la matrice de scores blosum. Ainsi, la matrice n'est initialisée qu'une seule fois lors de l'exécution. De plus, une fonction *getScore* propre à cette classe permet d'obtenir directement le score de la combinaison d'acides aminés d'intérêt.

3.4 Test sur différents ordinateurs

Après avoir testé notre projet avec le script *testfinal*, celui-ci répondait aux attentes. Cependant en le testant sur différents ordinateurs, une variabilité dans les résultats est apparue. En effet, les résultats de l'un des ordinateurs présentaient une variabilité allant jusqu'à 200% par rapport aux autres ordinateurs. Cela paraissait étrange, cependant il a été conclu que cette différence était sûrement due à des différences de performances de cet ordinateur étant donné que les valeurs des 3 autres avaient une moins grande variabilité entre eux, de l'ordre de 25%. Néanmoins, lorsque les tests étaient effectués sur un même ordinateur, ceux-ci donnaient des résultats très proches, de l'ordre de 1% de variabilité. Cela confirme donc bien la reproductibilité des tests.

4 Utilisation des intelligences artificielles (IA)

Concernant l'utilisation des intelligences artificielles, celles-ci nous ont été utiles à plusieurs reprises lors de notre projet. Lors de la partie préliminaire du projet, lorsque les fichiers C++ lisent un fichier en binaire avec la fonction *read*, celle-ci lit et stocke des chiffres en base 2 dans une variable. Certaines difficultés sont apparues lorsqu'il a fallu les convertir en base 10. Les IAs ont indiqués l'existence de la fonction *memcpy* utile pour effectuer la conversion d'un chiffre en base 2 à un chiffre en base 10 d'une variable à une autre.

Dans la seconde partie du projet, nous nous sommes servis des intelligences artificielles pour mieux comprendre en profondeur le fonctionnement de l'algorithme de Smith-Waterman modifié qui est expliqué dans les documents de références. Elles nous ont été utiles pour mieux comprendre le graphe de dépendance étant donné que celui-ci implique 3 matrices distinctes mais dépendantes. De plus, le document de référence mentionnait le fait de pouvoir réaliser l'algorithme en ne gardant en mémoire que 2 colonnes de matrice ainsi qu'un élément de la dernière matrice. L'IA a permis de mieux comprendre les explications fournies.

Notre dernière utilisation de celle-ci a permis de comprendre une erreur de segmentation. Lors de l'écriture du fichier *smith_waterman.cpp*, une erreur de segmentation est apparue. Celle-ci arrivait au moment du return de la fonction. Pour mieux comprendre d'où pouvait venir une telle erreur, l'IA nous a aidé en nous expliquant les potentielles causes comme le « Stack smashing », ce qui nous a permis de débayer l'algorithme.

5 Conclusion

En conclusion, l'exécution de notre projet donne des résultats satisfaisants. Nous avons opté pour un tri par insertion vu son efficacité dans le cas où le tableau est relativement petit et presque trié à chaque itération. Les protéines d'intérêt ont été stockées dans une « array » vu que nous n'avons pas besoin de l'aspect dynamique d'un vecteur ou d'une « multimap », ce qui optimise l'occupation mémoire. Une classe *BlosumMatrix* a été implémentée permettant d'alléger l'algorithme de Smith Waterman qui est une fonction déterminante dans le temps d'exécution de notre projet. Après plusieurs tests, nous obtenons des mesures reproductibles témoignant d'un algorithme stable. Finalement, tout ceci permet d'atteindre les objectifs mentionnés dans le cahier des charges et de réussir les tests finaux.